MEMORIA TÉCNICA — PUNTO Y APARTE

Buscador de libros y biblioteca personal
Proyecto individual del bootcamp Full Stack · Adrián Baile Campos
Tecnologías: HTML5, CSS3, JavaScript (módulos ES), `localStorage` y API de Open Library.

---

1. Descripción del proyecto

Punto y Aparte es una web de foro para lectores. Sobre el prototipo inicial en HTML y CSS se ha añadido interactividad con JavaScript: un **buscador de libros** conectado a la API pública de Open Library y una **biblioteca personal** donde el usuario guarda, ordena y elimina libros. La biblioteca se conserva en el navegador mediante `localStorage`.

1.1 Objetivos de aprendizaje

- Aplicar programación orientada a objetos con clases, métodos, composición y herencia.
- Separar la obtención de datos, la lógica y la construcción del DOM en módulos distintos.
- Consumir una API externa con `fetch` y `async/await`, gestionando errores.
- Actualizar la interfaz con eventos (`submit`, `click`, `change`) sin recargar la página.
- Guardar datos del usuario en `localStorage` y reconstruirlos como objetos.
- Trabajar con Git usando ramas y Pull Requests.

1.2 Alcance

Dentro del alcance	Fuera del alcance
Búsqueda por título, autor o ISBN en Open Library	Inicio de sesión y creación de cuenta (el formulario es visual)
Resultados separados en exactos y coincidentes	Recomendaciones de libros
Añadir y eliminar libros de la biblioteca personal	Opiniones, reseñas y comentarios dinámicos
Ordenar por fecha de inserción y por título (A-Z, Z-A)	Sincronizar la biblioteca entre dispositivos
Persistencia de la biblioteca en `localStorage`	Apartados *Leyendo ahora* y *Mis reseñas* del perfil (estáticos)

2. Requisitos

2.1 Requisitos funcionales

ID	Requisito	Dónde se implementa
RF-01	Buscar libros por título, autor o ISBN	`index.html`, `DOMController.onSearch`, `BookSearch.search`
RF-02	Avisar si el término tiene menos de 2 caracteres	`DOMController.onSearch`
RF-03	Separar resultados exactos y coincidentes	`SearchResult.classify`
RF-04	Mostrar cada libro con portada, título, autores y año	`BookCard.createBookCard`
RF-05	Mostrar portada por defecto si el libro no tiene	`Book.getCoverUrl`, evento `error` en `BookCard`
RF-06	Añadir un libro a la biblioteca sin duplicados	`Library.add`, `Library.has`, `DOMController.onAdd`
RF-07	Eliminar un libro de la biblioteca	`Library.remove`, `DOMController.renderLibrary`
RF-08	Ordenar la biblioteca por fecha y por título	`Library.sort`, selector `#sortSelect`
RF-09	Guardar y cargar la biblioteca en `localStorage`	`Library.save`, `Library.load`
RF-10	Mostrar mensajes de carga, sin resultados y error	`DOMController.showMessage`

2.2 Requisitos no funcionales

ID	Criterio	Solución
RNF-01	JavaScript modular sin frameworks	Archivos ES con `import` / `export` y un único punto de entrada, `main.js`.
RNF-02	Código separado por responsabilidades	Carpetas `models/`, `services/`, `components/`, `ui/` y `utils/`.
RNF-03	Diseño adaptable	Mobile First con una media query a partir de 768px. Las tarjetas se colocan en cuadrícula con `auto-fill`.
RNF-04	Mismo aspecto que el prototipo	Las tarjetas creadas con JS reutilizan las clases CSS `.book`, `.book__cover` y `.author`.
RNF-05	Ejecución local	Los módulos ES requieren un servidor local, por ejemplo Live Server.

2.3 Historias de usuario

- **HU-01:** Como lector, quiero buscar un libro por título, autor o ISBN para encontrarlo sin salir de la web.
- **HU-02:** Como lector, quiero ver primero los resultados exactos para localizar rápido el libro que busco.
- **HU-03:** Como lector, quiero guardar libros en mi biblioteca para tenerlos a mano más tarde.
- **HU-04:** Como lector, quiero ordenar mi biblioteca y quitar libros para mantenerla organizada.

---

3. Reglas de negocio

ID	Regla
RN-01	El término de búsqueda debe tener al menos 2 caracteres.
RN-02	En la búsqueda por ISBN se eliminan guiones y espacios antes de llamar a la API.
RN-03	Un resultado es **exacto** si su título (búsqueda por título) o alguno de sus autores (búsqueda por autor) coincide con el término, sin distinguir mayúsculas ni tildes. En la búsqueda por ISBN todos los resultados son exactos.
RN-04	Un libro se identifica por su `id` (la clave `/works/...` de Open Library sin el prefijo) y no puede añadirse dos veces.
RN-05	Al añadir un libro se guarda la fecha actual en `addedDate`, que se usa para ordenar por fecha.
RN-06	Ordenar no modifica el orden guardado: `Library.sort` devuelve una copia ordenada.

---

4. API y mapeo de datos

4.1 Open Library

Uso	Petición
Buscar por título	`https://openlibrary.org/search.json?title={término}&limit=20`
Buscar por autor	`https://openlibrary.org/search.json?author={término}&limit=20`
Buscar por ISBN	`https://openlibrary.org/search.json?isbn={isbn}&limit=20`
Portada	`https://covers.openlibrary.org/b/id/{cover_i}-M.jpg`

La API es gratuita, no necesita clave y permite llamadas desde el navegador. Toda la comunicación con ella está en `OpenLibraryApiService.js`, que exporta la función `getBooks(input, inputType)`. El término se codifica con `encodeURIComponent` para que espacios, tildes o `&` no rompan la URL. Si la respuesta no es correcta, se lanza `FailureDoingFetchError`.

Las portadas no se piden con `fetch`. `Book.getCoverUrl()` construye la URL y se asigna a `img.src`, de modo que el navegador las descarga directamente.

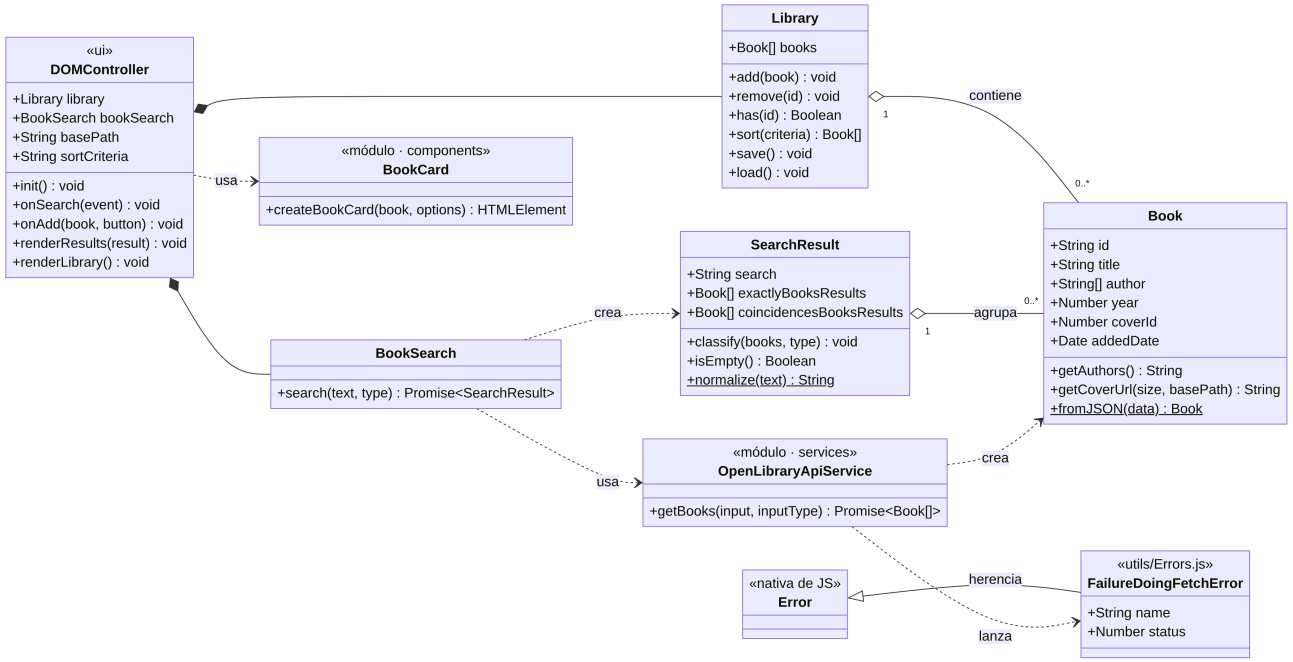
4.2 Mapeo JSON → `Book`

Campo de la API	Propiedad de `Book`	Transformación
`key`	`id`	Se quita el prefijo `/works/`.
`title`	`title`	Se conserva.
`author_name`	`author`	Array de nombres. `getAuthors()` los une con comas o devuelve "Autor desconocido".
`first_publish_year`	`year`	Se conserva. Si falta, se muestra "Año desconocido".
`cover_i`	`coverId`	Se usa para construir la URL de la portada.
—	`addedDate`	Se asigna al añadir el libro a la biblioteca.

---

5. Diseño del modelo

5.1 Diagrama de clases



- **Composición:** `DOMController` crea y contiene una `Library` y un `BookSearch`.
- **Agregación:** `Library` contiene libros (`Book`) y `SearchResult` agrupa los libros encontrados.
- **Herencia:** `FailureDoingFetchError` hereda de `Error`.
- `OpenLibraryApiService` y `BookCard` son **módulos de funciones**, no clases.

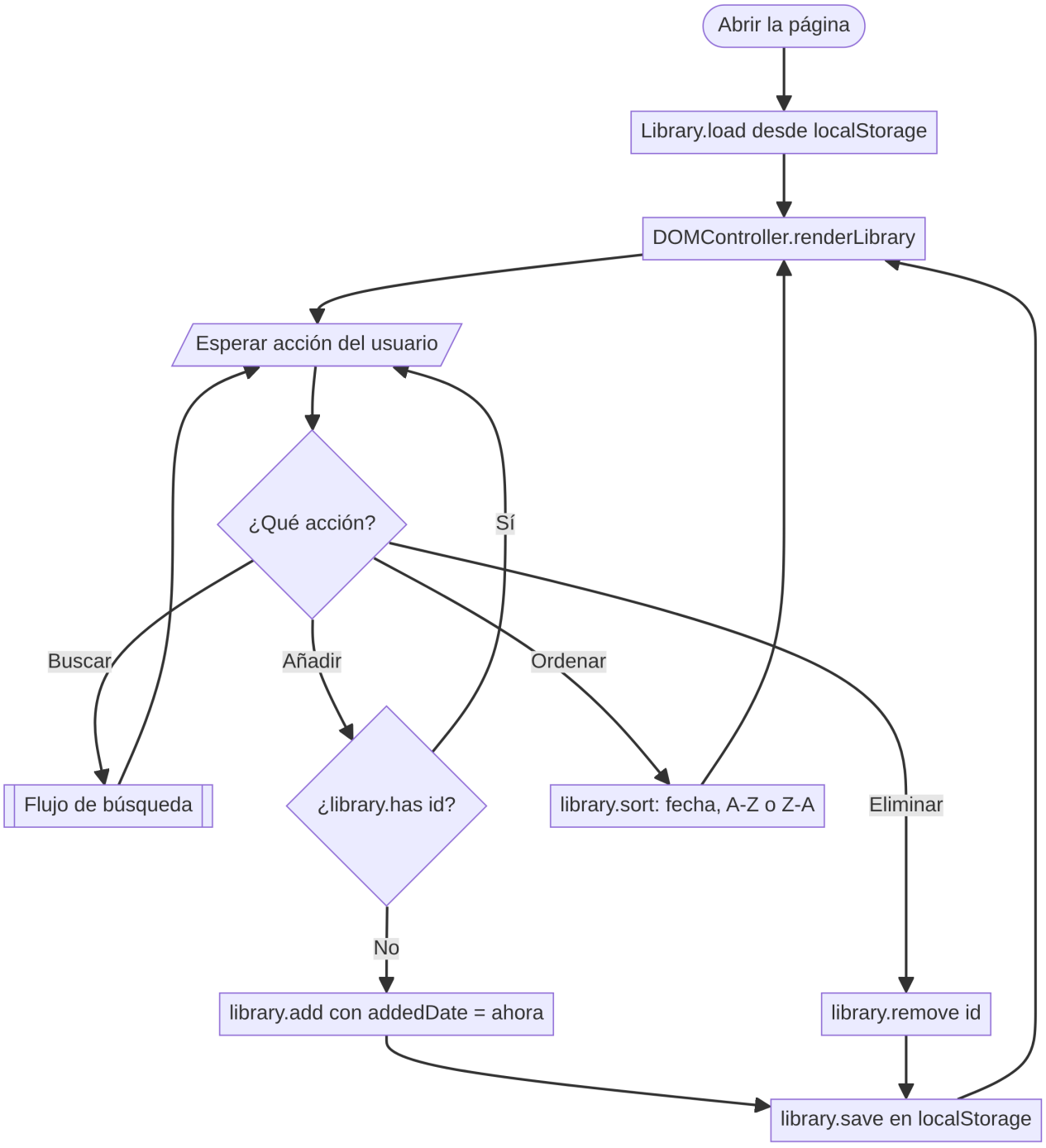
En la planificación inicial se identificaron cinco clases: Libro, Autor, Biblioteca, BuscadorLibro y ResultadoBusqueda. Durante el desarrollo se eliminó **Autor**, porque la API ya devuelve los autores como un array de textos y basta con una propiedad de `Book`. También se pasaron los nombres al inglés para mantener la coherencia con el resto del código.

5.2 Organización por capas

Capa	Archivos	Responsabilidad
Servicio	`services/OpenLibraryApiService.js`	Única parte que hace `fetch`. Convierte la respuesta en objetos `Book`.
Modelo	`models/Book.js`, `Library.js`, `BookSearch.js`, `SearchResult.js`	Datos y lógica. No tocan el DOM.
Componente	`components/BookCard.js`	Crea la tarjeta HTML de un libro con un botón configurable (Añadir o Eliminar).
Interfaz	`ui/DOMController.js`	Registra eventos y pinta resultados y biblioteca.
Utilidades	`utils/Errors.js`	Error personalizado para los fallos de `fetch`.
Entrada	`main.js`	Calcula `basePath` según la página y arranca `DOMController`.

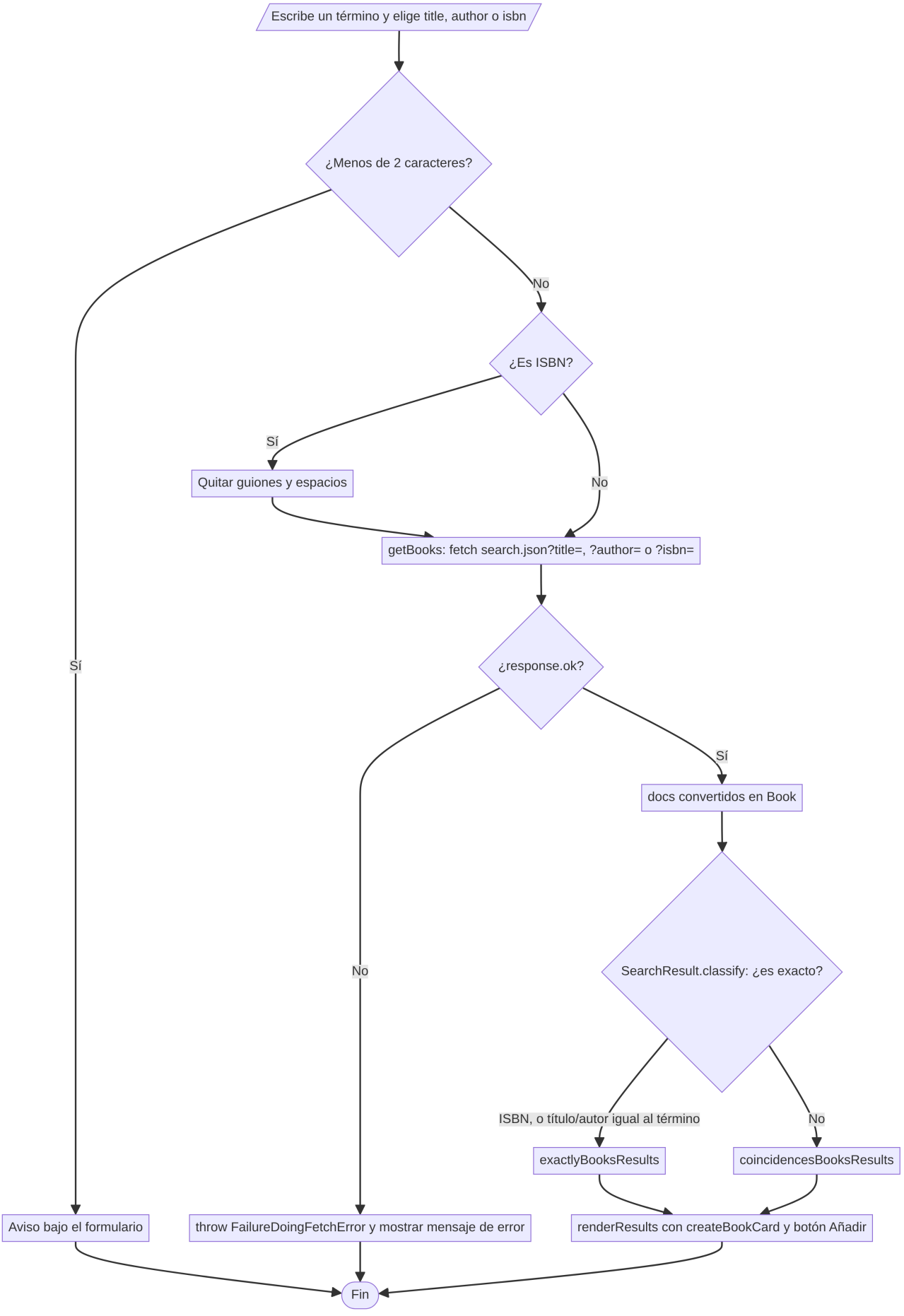
6. Recorrido de la aplicación

6.1 Flujo general

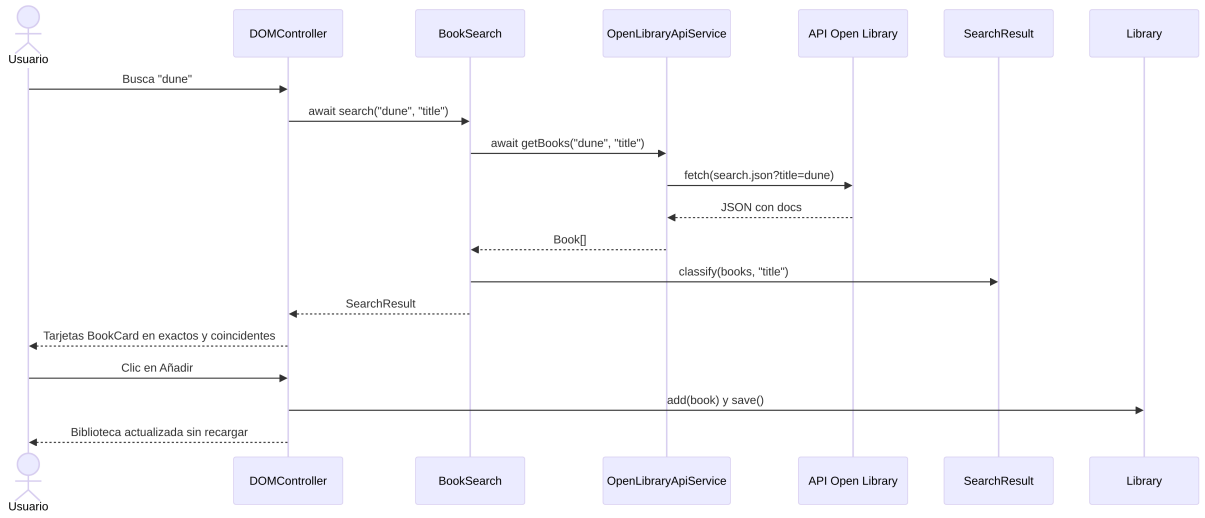


Las dos páginas cargan el mismo `main.js`. `DOMController.init()` carga la biblioteca y solo activa lo que existe en cada página: el buscador en `index.html` y *Mi biblioteca* en `pages/profile.html`. Como `localStorage` es común a todo el sitio, los libros añadidos en Inicio aparecen en el Perfil.

6.2 Flujo de búsqueda



6.3 Flujo de datos entre clases



6.4 Persistencia

Clave de `localStorage`	Contenido
`pya-library`	Array de libros en JSON: `id`, `title`, `author`, `year`, `coverId` y `addedDate`.

`JSON.parse` devuelve objetos planos, sin los métodos de la clase. Por eso `Library.load()` reconstruye cada elemento con `Book.fromJSON()`, que además convierte `addedDate` de texto a `Date`. Los accesos a `localStorage` van dentro de `try/catch` para que un fallo del navegador no rompa la página.

---

7. Pruebas

Pruebas de aceptación propuestas para comprobar en el navegador con Live Server.

Nº	Acción	Resultado esperado
P-01	Buscar "a"	Aviso de mínimo 2 caracteres y sin llamada a la API
P-02	Buscar el título "El principito"	El libro con ese título exacto aparece en Exactos y las demás ediciones en Coincidentes
P-03	Buscar por autor "Gabriel García Márquez"	Sus libros aparecen en Exactos
P-04	Buscar por ISBN con guiones	Se limpian los guiones y el resultado aparece en Exactos
P-05	Pulsar Añadir	El botón cambia a "En tu biblioteca" y queda desactivado
P-06	Ir a Perfil	El libro aparece con su fecha y el contador se actualiza
P-07	Cambiar el orden a A-Z y a Z-A	La lista se reordena sin recargar
P-08	Eliminar un libro y recargar la página	El libro sigue eliminado
P-09	Buscar sin conexión	Mensaje de error de conexión
P-10	Abrir en móvil y escritorio	Formulario y tarjetas se adaptan sin scroll horizontal

8. Gestión del trabajo

El proyecto se ha desarrollado de forma individual, con Git y GitHub (`adribai/punto-y-aparte`). Cada parte se trabajó en su propia rama y se integró en `main` mediante Pull Request (13 en total).

Rama	Contenido
`Models`	Clases `Book`, `Library`, `BookSearch` y `SearchResult` y sus métodos
`api`	Servicio de Open Library y error personalizado
`DOMComponents`	Componente `BookCard`
`DOMController`	Controlador de la interfaz y `main.js`
`styles`	Estilos del buscador, las tarjetas y *Mi biblioteca*
`featMain`	Integración en el HTML y búsqueda por ISBN
`hotfix/style`	Correcciones de estilos

Fecha	Hito
13/10/2026	Prototipo inicial en HTML y CSS
29/10/2026	Planificación: funciones, clases y diagramas
03/11/2026	Primeras clases, error personalizado y servicio de la API
04/11/2026	Simplificación del servicio, eliminación de la clase Autor y del servicio de portadas
05/11/2026	Métodos de los modelos, `DOMController`, `main.js` y estilos
06/11/2026	Búsqueda por ISBN, ajustes de estilo y documentación

Tablero de tareas: *añadir aquí el enlace a Trello o al tablero utilizado.*

9. Conclusiones

9.1 Desafíos y soluciones

Desafío	Solución
Los textos con espacios, tildes o `&` rompían la URL.	`encodeURIComponent()` sobre el término.
Decidir si un resultado es exacto con mayúsculas y tildes distintas.	`SearchResult.normalize()` pasa a minúsculas y quita las tildes antes de comparar.
Las portadas se pedían con `fetch` y `blob`, lo que era innecesario.	Se asigna la URL a `img.src`. Si no hay portada o falla la carga, se usa una imagen por defecto.
Las rutas relativas de imágenes cambian entre `index.html` y `pages/`.	`main.js` calcula `basePath` (`""` o `"../"`) y se pasa hasta `getCoverUrl`.
`JSON.parse` pierde los métodos de `Book`.	`Book.fromJSON()` reconstruye cada libro al cargar.
La clase Autor no aportaba nada.	Se eliminó y los autores son una propiedad de `Book`.
Un error de sintaxis en un módulo impedía cargar toda la aplicación (exportación duplicada, export olvidado).	Revisar la consola del navegador y mantener un único `export` por función.
Los módulos ES no cargan al abrir el HTML con doble clic.	Ejecutar con Live Server, como indica el README.

9.2 Posibles mejoras

1. Pedir solo los campos necesarios a la API con el parámetro `fields` para reducir la respuesta.
2. Guardar en `localStorage` también el criterio de orden elegido.
3. Añadir un filtro de texto dentro de *Mi biblioteca*.
4. Exportar la biblioteca a CSV.
5. Añadir modo oscuro.
6. Implementar inicio de sesión simulado y reseñas propias sobre los libros guardados.

9.3 Conclusión

El proyecto aplica la programación orientada a objetos, la manipulación del DOM y el consumo de una API real sobre un prototipo existente. Separar el servicio, los modelos, el componente y el controlador ha permitido trabajar cada parte en una rama distinta y localizar los errores con facilidad. Por ejemplo, la tarjeta `BookCard` se reutiliza tanto en los resultados como en la biblioteca cambiando solo el botón. La biblioteca en `localStorage` cubre la persistencia sin necesidad de un backend.